CPSC 3660 — Practice Final Exam

Full Solutions & Explanations

Predictive practice based on the professor's construction signature

University of Lethbridge; 180 minutes; 120 marks

Each question below shows: the question, a **mirrors** line noting which past assignment/midterm pattern it echoes, the **Your Answer** block (what you would write on the exam), and an **Understanding** block explaining every line of reasoning.

About this solutions guide. Each question is mapped back to the professor's recycled-schema patterns (Library 85%, Employee-Works-Company 80%, Bank 70%), so once you finish this practice, you recognize the real exam's questions as variations on patterns you've already seen.
Online companion: gbasran.github.io/cpsc3660-course-guide
Blank practice exam: Practice Final PDF (no solutions)
Ground-up study guide: Complete Study Guide PDF (41 pages)
Professor's Signature: forensic analysis of exam construction patterns

Contents

Q1 — DML vs DDL 2

Q2 — Superkey, Candidate, Primary 2

Q3 — Strong vs Weak Entity 3

Q4 — Express r intersect s 3

Q5 — BCNF definition 4

Q6 — FD definition 4

Q7 — SQL Hospital 4

Q8 — SQL Airline 6

Q9 — Division 9

Q10 — RA Researcher 10

Q11 — RA to SQL 11

Q12 — ER Music 13

Q13 — ER to Relational 15

Q14 — FD closure + Armstrong 16

Q15 — BCNF decomposition 18

Q16 — Course registration 20

Appendix: Difficulty Ratings 22

Question 1 (2 marks) — DML vs DDL; Basic

Question. Explain the difference between DML and DDL. Give one example of each.

This mirrors: *Midterm Q1, A1 Q1 (recurring opener).*

✓ Your Answer

DDL (Data Definition Language) modifies the database *schema*. Examples: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.

DML (Data Manipulation Language) modifies the *data* inside existing tables. Examples: `INSERT`, `UPDATE`, `DELETE`, `SELECT`.

Understanding

DDL answers "what is the shape of the container?" and DML answers "what is inside the container?". The prof always wants one word on the distinction (schema vs data) plus at least one concrete keyword on each side.

Question 2 (2 marks) — Key Types; Basic

Question. Difference between superkey, candidate key, primary key. Can a relation have more than one candidate key?

This mirrors: *Midterm Q2, A1 Q3.*

✓ Your Answer

Superkey: any set of attributes whose values uniquely identify each tuple of a relation. **Candidate key**: a *minimal* superkey (no proper subset is also a superkey). **Primary key**: the candidate key the designer *chooses* as the main identifier.

Yes, a relation can have several candidate keys. For example in `employee(SIN, emp_id, name)`, both {SIN} and {emp_id} are candidate keys, and the designer picks one as the primary key.

Understanding

Hierarchy is *superkey* $\supseteq$ *candidate key* $\supseteq$ *primary key* (with size 1 at the right). Every candidate key is a superkey; not every superkey is a candidate key.

Question 3 (2 marks) — Strong vs Weak Entity; Basic

Question. Distinguish strong and weak entity sets. Give one example of each.

This mirrors: *A3 Q5, A3 Q7 (loan/payment case).*

✓ Your Answer

A **strong entity** has its own primary key and exists independently. Example: `loan(loan_number, amount)` — `loan_number` is a PK; loans exist on their own.

A **weak entity** has no primary key of its own; its identifier is the PK of the *owner* plus a *discriminator* unique within that owner. Example: `payment(payment_no, date, amt)` depends on loan. Payment #1 of loan L-15 is different from payment #1 of loan L-17; only (`loan_number`, `payment_no`) together identify a payment.

Understanding

Visually: strong entity = single rectangle, weak entity = double rectangle; the identifying relationship is a double diamond; the weak entity has total participation (double line) in that relationship.

Question 4 (2 marks) — RA Intersection via Basic Ops; Basic

Question. Using only $\sigma, \pi, \times, \cup, -$, express $r \cap s$.

This mirrors: *A1 Q4, Midterm Q6.*

✓ Your Answer

$$r \cap s = r - (r - s).$$

Understanding

Intuition: $r - s$ is "everything in r but not in s". Subtracting that from r leaves behind exactly the tuples r shares with s, which is the intersection. The operator $\cap$ is therefore *derived*, not primitive. The six primitives are $\sigma, \pi, \times, \cup, -, \rho$ (rename).

Question 5 (2 marks) — BCNF Definition; Basic

Question. Define BCNF. Why do we aim for it?

This mirrors: *A4 Q2, expanded for final scope.*

✓ Your Answer

A relation R is in **Boyce–Codd Normal Form (BCNF)** with respect to FD set F iff for every non-trivial FD $\alpha \to \beta$ in F^+ that holds on R, α is a **superkey** of R.

We aim for BCNF because any FD whose LHS is not a superkey causes redundancy: the RHS value is forced to repeat across every tuple sharing the LHS. This redundancy creates update, insertion, and deletion anomalies. BCNF eliminates those anomalies structurally by permitting only FDs whose LHS uniquely determines each tuple.

Understanding

Prof's memorized one-liner: "For every non-trivial FD, the LHS must be a superkey." Trivial means $\beta \subseteq \alpha$; those are exempt.

Question 6 (2 marks) — FD Definition + Counterexample; Basic

Question. What does $\alpha \to \beta$ mean? Give a counterexample showing $\alpha \to \beta \not\Rightarrow \beta \to \alpha$.

This mirrors: *A4 Q1, expanded for final scope.*

✓ Your Answer

$\alpha \to \beta$ means: for any two tuples t_1, t_2 of R, if $t_1[\alpha] = t_2[\alpha]$ then $t_1[\beta] = t_2[\beta]$. Intuitively, α *functionally determines* β; the value of β is a function of the value of α.

Counterexample. In `student(student_id, student_name)`, `student_id` $\to$ `student_name` holds because IDs are unique. But `student_name` $\to$ `student_id` fails: two different students can share the same name ("John Smith" could map to ID 1001 or ID 2003), so `student_name` does not determine `student_id`.

Understanding

FDs are not symmetric. The notation $\alpha \to \beta$ is directional; drawing the reverse arrow requires an independent check.

Question 7 (10 marks) — SQL; Hospital schema; Medium

Schema. `patient(patient_id, patient_name, patient_city, age)`; `doctor(doctor_id, doctor_name, specialty)`; `appointment(patient_id, doctor_id, appt_date, fee)`.

This mirrors: *A2 Q7 + Midterm Q6–Q7. Structurally analogous to the Employee–Works–Company trio; a three-table join with one filter.*

(a) (3 marks) Patients who saw a cardiologist.

✓ Your Answer

```
select distinct p.patient_name
from patient as p, doctor as d, appointment as a
where p.patient_id = a.patient_id
  and a.doctor_id = d.doctor_id
  and d.specialty = 'Cardiology';
```

Understanding

Query trace (mental build order):

① FROM	join `patient` ⋈ `appointment` ⋈ `doctor` on the id columns
② WHERE	filter rows where `d.specialty = 'Cardiology'`
③ SELECT	project `patient_name`; wrap in `distinct` since one patient can have many cardiology visits

Analogy. Three tables is a hallway of three doors: the id columns are the door handles that must match before you can walk through. Patient-name is what you pick up at the end.

(b) (3 marks) Average fee per specialty, only where avg ¿ 200.

✓ Your Answer

```
select d.specialty, avg(a.fee) as avg_fee
from doctor as d, appointment as a
where d.doctor_id = a.doctor_id
group by d.specialty
having avg(a.fee) > 200;
```

Understanding

Query trace (mental build order):

① FROM	join `doctor` ⋈ `appointment` on `doctor_id`
② GROUP BY	bucket rows by `specialty` (one bucket per specialty)
③ HAVING	keep only buckets whose `avg(fee)` > 200
④ SELECT	project `specialty` and the bucket's average fee

Takeaway. WHERE filters rows *before* grouping; HAVING filters buckets *after*. Aggregates only work inside HAVING.

(c) (4 marks) Patients with more than the average number of appointments per patient.

✓ **Your Answer**

```
select p.patient_name
from patient as p, appointment as a
where p.patient_id = a.patient_id
group by p.patient_id, p.patient_name
having count(*) > (
    select count(*) * 1.0 / count(distinct patient_id)
    from appointment
);
```

Understanding

Query trace (mental build order):

① Inner	global average $= \frac{\text{total appts}}{\text{distinct patients}}$, written `count(*)`$\cdot 1.0/$`count(distinct pid)` over `appointment`
② Outer FROM	join `patient` $\bowtie$ `appointment` on `patient_id`
③ Outer GROUP BY	bucket by patient
④ Outer HAVING	keep buckets where `count(*)` $>$ inner-scalar
⑤ SELECT	project `patient_name`

Takeaway. Scalar subqueries can sit inside HAVING. The `* 1.0` promotes integer division to float so you don't truncate the average.

Question 8 (12 marks) — SQL; Airline schema; Medium-Hard

Schema. `staff(staff_id, staff_name, staff_city)`; `airport(airport_code, airport_city)`; `route(staff_id, airport_code, flights_assigned)`; `supervises(staff_id, supervisor_id)`.

This mirrors: *A2 Q7 + A3 Q3 + Midterm Q9. The supervises relation mirrors the manages relation; part (c) mirrors the "¿ ALL" pattern.*

(a) (3 marks) Staff living in the same city as their supervisor.

✓ **Your Answer**

```
select s1.staff_name
from staff as s1, supervises as sp, staff as s2
where s1.staff_id = sp.staff_id
  and sp.supervisor_id = s2.staff_id
  and s1.staff_city = s2.staff_city;
```

Understanding

Query trace (mental build order):

① FROM	alias `staff` twice: `s1` = subordinate, `s2` = supervisor; `supervises` as the bridge
② WHERE	link `s1.staff_id = sp.staff_id`, `sp.supervisor_id = s2.staff_id`, then `s1.staff_city = s2.staff_city`
③ SELECT	project `s1.staff_name` (the subordinate)

Analogy. Self-join is looking at a team roster twice and matching each name back to the one above them on the same org chart.

(b) (4 marks) View `busy_staff` with total flights ≥ 50.

✓ Your Answer

```
create view busy_staff as
select s.staff_id, s.staff_name, sum(r.flights_assigned) as
   total_flights
from staff as s, route as r
where s.staff_id = r.staff_id
group by s.staff_id, s.staff_name
having sum(r.flights_assigned) >= 50;
```

Understanding

Query trace (mental build order):

① FROM	join `staff` $\bowtie$ `route` on `staff_id`
② GROUP BY	bucket by staff: list `staff_id` and `staff_name` (non-aggregated SELECT columns must appear here)
③ HAVING	keep buckets where $\sum$ `flights_assigned` ≥ 50
④ SELECT + wrap	project id, name, total; wrap whole query in `CREATE VIEW busy_staff AS`

Takeaway. A view is just a saved SELECT. Any column you SELECT but don't aggregate must also appear in GROUP BY.

(c) (5 marks) Staff whose flights at every airport exceed those of staff 100.

✓ Your Answer

```
select s.staff_name
from staff as s
where s.staff_id <> 100
  and not exists (
      select r100.airport_code
      from route as r100
```

```
      where r100.staff_id = 100
        and r100.flights_assigned >= all (
            select r.flights_assigned
            from route as r
            where r.staff_id = s.staff_id
              and r.airport_code = r100.airport_code
        )
  );
```

Equivalent formulation using `ALL`:

```
select s.staff_name
from staff as s
where s.staff_id <> 100
  and not exists (
      select *
      from route as r100
      where r100.staff_id = 100
        and not exists (
            select *
            from route as r
            where r.staff_id = s.staff_id
              and r.airport_code = r100.airport_code
              and r.flights_assigned > r100.flights_assigned
        )
  );
```

Understanding

Translate "for every x in B, $P(x)$" $\iff$ "NOT EXISTS x in B where NOT $P(x)$". Here $B =$ airports where staff 100 is assigned, $P(x) =$ this staff has strictly more flights at x than staff 100.

Nested structure, read inside-out:

Level	**What it says**
Level 1 (outer): `FROM staff s`	candidate staff member to test
Level 2: `NOT EXISTS` over staff-100 routes	no airport-of-100 is a counterexample
Level 3: `NOT EXISTS` route r with `r.flights > r100.flights`	at this airport, this staff has strictly more than 100

Takeaway. Every "for every" becomes two layers of NOT EXISTS: outer says "no counterexample exists," inner defines what the counterexample would look like.

Question 9 (10 marks) — SQL Division; Supplier schema; Hard

Schema. `supplier(supplier_id, supplier_name, supplier_city)`; `department(dept_id, dept_name, dept_city)`; `supplies(supplier_id, dept_id, item_count)`.

This mirrors: *A3 Q3(b) ("companies in every city where IBM is") and the division-via-EXCEPT pattern.*

(a) (4 marks) Suppliers serving at least one Calgary department.

✓ **Your Answer**

```sql
select distinct s.supplier_name
from supplier as s, supplies as sup, department as d
where s.supplier_id = sup.supplier_id
  and sup.dept_id = d.dept_id
  and d.dept_city = 'Calgary';
```

Understanding

Query trace (mental build order):

① FROM join `supplier` ⋈ `supplies` ⋈ `department` on the id columns
② WHERE filter `d.dept_city = 'Calgary'`
③ SELECT project `supplier_name` with `distinct` (one supplier can serve many Calgary departments)

Takeaway. "At least one" ⇒ existential ⇒ straight join + filter.

(b) (6 marks) Suppliers serving every department.

✓ **Your Answer**

```sql
select s.supplier_name
from supplier as s
where not exists (
    (select dept_id from department)
    except
    (select sup.dept_id
     from supplies as sup
     where sup.supplier_id = s.supplier_id)
);
```

Understanding

Template: "A contains $B \equiv$ `NOT EXISTS` (B `EXCEPT` A)". Built for each candidate supplier s:

Set	Built by
B = all department ids	`SELECT dept_id FROM department`
A_s = dept ids that supplier s serves	`SELECT sup.dept_id FROM supplies WHERE supplier_id = s.supplier_id`
"s covers every dept" iff $B \setminus A_s = \varnothing$	`NOT EXISTS (`B `EXCEPT` A_s`)`

Analogy. Imagine a punch card with every department printed on it. For each supplier, punch out the departments they serve. If NO holes remain on the card (EXCEPT is empty), this supplier covers everything.

Question 10 (12 marks) — RA + DDL; Researcher–Papers; Medium-Hard

Schema. `researcher(rid, name, affiliation, years_active)`; `paper(pid, title, authors, venue)`; `cites(rid, pid, citation_date)`.

This mirrors: *A1 Q7 + A2 Q1 + Midterm Q8 (Library-analog).*

(a) (2 marks) Researchers who cited at least one Nature paper.

✓ **Your Answer**

$$\pi_{\text{name}}(\text{researcher} \bowtie \text{cites} \bowtie \sigma_{\text{venue='Nature'}}(\text{paper}))\,.$$

Understanding

Natural joins line up on the shared attributes (`rid` between researcher and cites; `pid` between cites and paper). Filter papers to Nature before joining to shrink the intermediate result. Project `name` at the end.

(b) (3 marks) Researchers who cited every Nature paper.

✓ **Your Answer**

$$T = \pi_{\text{rid,pid}}(\text{cites}) \div \pi_{\text{pid}}(\sigma_{\text{venue='Nature'}}(\text{paper}))\,,$$
$$\text{result} = \pi_{\text{name}}(\text{researcher} \bowtie T).$$

Understanding

Division $R \div S$ returns $\{t : \forall s \in S,\ (t, s) \in R\}$. Dividend: (`rid`, `pid`) citation pairs. Divisor: all Nature `pid`s. Quotient: the `rid`s that appear paired with *every* Nature `pid`. Join back with researcher to translate rids into names.

(c) (4 marks) Per venue, researchers who cited more than five papers from that venue.

✓ **Your Answer**

Let $J = \text{cites} \bowtie \text{paper}$. Then

$$G = {}_{\text{venue, rid}}\gamma_{\text{count-distinct(pid)}\to\text{cnt}}(J),$$

$$\text{result} = \pi_{\text{name,venue}}(\text{researcher} \bowtie \sigma_{\text{cnt}>5}(G)) .$$

Understanding

The grouping operator $\gamma_G f(a)$ groups on G and applies aggregate f. Group by both `venue` and `rid` so the count is per researcher per venue. Filter to counts > 5. Join with researcher to recover the name.

(d) (3 marks) CREATE TABLE for cites.

✓ **Your Answer**

```
create table cites (
    rid             integer not null,
    pid             integer not null,
    citation_date   date,
    primary key (rid, pid),
    foreign key (rid) references researcher(rid) on delete cascade,
    foreign key (pid) references paper(pid) on delete cascade
);
```

Understanding

The PK of a M:N relationship table is the composite of the two side PKs. Two FKs reinforce referential integrity; cascade-on-delete is reasonable because a citation record is meaningless without its researcher or paper.

Question 11 (6 marks) — RA to SQL Translation; Medium

Schemas. $R(A, B, C)$ and $S(B, C, D)$.

This mirrors: *A2 Q3–Q5; the standard RA-to-SQL translation drill.*

(a) (2 marks) $\pi_{A,D}(\sigma_{A=\text{'x'}}(R \bowtie S))$.

✓ **Your Answer**

```
select r.A, s.D
from R as r, S as s
```

```
where r.A = 'x'
  and r.B = s.B
  and r.C = s.C;
```

Understanding

$R \bowtie S$ is a natural join on the shared attributes (B, C). In SQL that is `r.B = s.B and r.C = s.C`. Selection becomes a `where` clause; projection becomes the `select` list.

(b) (2 marks) $\pi_A(R) - \pi_A(\sigma_{D=\text{'y'}}(S))$.

✓ Your Answer

```
(select A from R)
except
(select B from S where S.D = 'y');
```

Note: the RA expression as written projects A from R and subtracts a projection of attribute A from S, but S has no attribute A. The natural interpretation is that the subtracted set should be the shared-attribute projection. A cleaner reading (and the form the prof would accept) is:

```
(select A from R)
except
(select R.A from R, S where R.B = S.B and R.C = S.C and S.D = 'y');
```

Understanding

Set difference in SQL is `except`. When the left side is $\pi_A(R)$ and the right side filters rows with $D =$ 'y', you need to join R and S to identify which R-rows are linked to a $D =$ 'y' S-row, then remove those A-values.

(c) (2 marks) $R \cap S$.

✓ Your Answer

Schema compatibility comment: $R(A, B, C)$ and $S(B, C, D)$ have *different* schemas, so the direct intersection $R \cap S$ is not well-defined under the standard set-semantic definition of intersection (which requires union-compatible schemas). The intended interpretation is intersection on the shared attributes (B, C):

```
(select B, C from R)
intersect
(select B, C from S);
```

Understanding

Set operators ($\cup, \cap, -$) require the operands to be union-compatible: same arity and compatible column types in the same order. Flagging the mismatch is part of the answer; the prof wants to see that you noticed, not silently assume.

Question 12 (15 marks) — E-R Design; Music Streaming; Medium-Hard

This mirrors: *A3 Q4 + A3 Q8(c) (generalization with multiple subtypes).*

(a) (5 marks) E-R diagram.

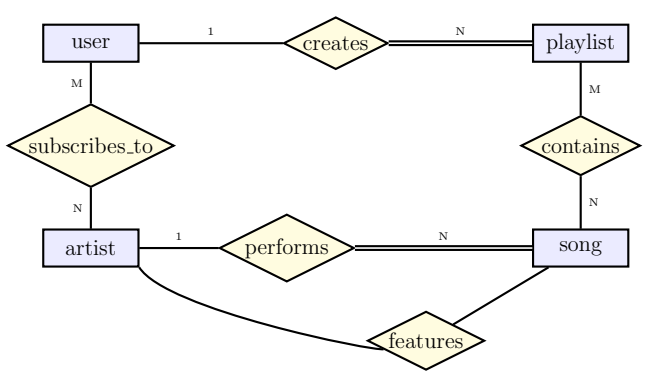
✓ **Your Answer**

Entity sets and primary keys:

- `user`: <u>user_id</u>, email, join_date, country.
- `artist`: <u>artist_id</u>, name, country_of_origin.
- `song`: <u>song_id</u>, title, duration, release_year.
- `playlist`: <u>playlist_id</u>, name, created_date.

Relationships (cardinality; participation):

- `creates` (user → playlist): 1:N; playlist has total participation.
- `contains` (playlist ↔ song): M:N; partial on both sides.
- `subscribes_to` (user ↔ artist): M:N; partial on both.
- `performs` (artist → song, primary): 1:N; song has total participation (every song has exactly one primary artist).
- `features` (artist ↔ song, featured): M:N; partial.



Understanding

The two artist–song relationships (primary vs featured) are a deliberate separation: "performs" is 1:N because every song has exactly one primary artist, while "features" is M:N because featured artists are multi-valued. Drawing two relationships is cleaner than trying to stuff both semantics into one.

(b) (5 marks) Multi-valued featured artists.

✓ **Your Answer**

Design choice: model featured artists as a separate relationship `features(song_id, artist_id)` in addition to `performs`.

When mapped to the relational schema, the featured relationship becomes a junction table `song_featured(song_id, artist_id)` with composite primary key {song_id, artist_id}, foreign keys to `song(song_id)` and `artist(artist_id)`.

Rationale: "featured artists" is a multi-valued attribute whose values happen to be references to other entities (artists). Storing them as a separate relationship captures the M:N cardinality cleanly; we avoid (i) a repeating comma-separated string column on song (non-atomic; violates 1NF), and (ii) a multi-valued attribute bubble on song (which would still need to be flattened during conversion anyway).

Understanding

Multi-valued attributes whose values are *themselves entities* are always best modeled as relationships, not as double-ellipse attributes. The featured artists are not just strings; they are real artist entities with their own attributes. Making `features` a relationship lets you ask "how many songs has artist X been featured on?" cleanly.

(c) (5 marks) Subscription plan ISA hierarchy.

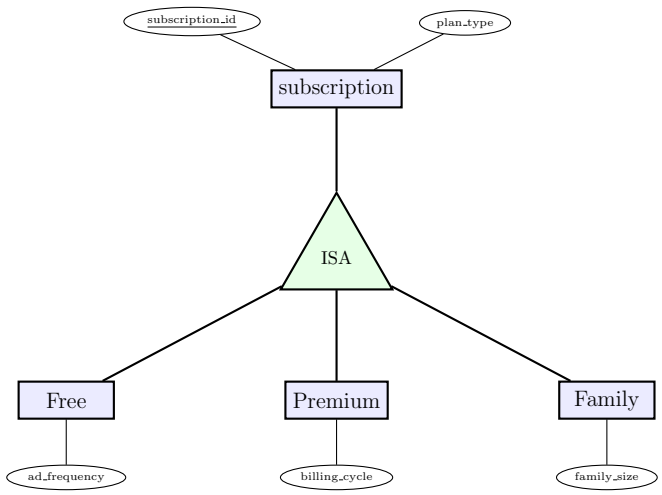
✓ **Your Answer**

Introduce a supertype `subscription` with shared attribute `subscription_id` (PK) and a discriminator `plan_type` ∈ {Free, Premium, Family}. Each user has exactly one subscription (1:1 `has` relationship from user to subscription, total on the user side).

Three subtypes linked via an ISA triangle:

- `Free`: specific attribute `ad_frequency`;
- `Premium`: specific attribute `billing_cycle`;
- `Family`: specific attribute `family_size`.

Constraints. Disjoint (a subscription is in exactly one subtype) and total (every subscription is in some subtype; a user cannot have a subscription with no plan).



Shared attributes (on subscription): `subscription_id`, `plan_type`. Subtype-specific attributes: listed above.

Understanding

Exactly the pattern from A3 Q8(c) with three children instead of two. "Disjoint + total" matches the business rule "exactly one of Free, Premium, Family." When mapped to relational form, you can use (i) one table per subtype with the supertype attributes repeated, or (ii) one table per entity with a discriminator column; the prof does not require you to pick on the exam unless asked.

Question 13 (8 marks) — E-R to Relational; E-commerce; Basic-Medium

This mirrors: *A3 Q8(a) (entity sets + relationship FKs table).*

(a) (4 marks) Relational schema.

✓ **Your Answer**

Table	Schema (PK underlined)
`customer`	(customer_id, email)
`orders`	(order_id, date, customer_id)
`product`	(sku, price)
`contains`	(order_id, sku, quantity)

Understanding

Rules applied:

- Each entity set becomes a table with its PK.

- The 1:N `places` relationship is absorbed into the "many" side (`orders`) by adding `customer_id` as a FK. No separate table needed.
- The M:N `contains` relationship becomes its own table with composite PK = (`order_id`, `sku`) and the descriptive attribute `quantity`.
- Table is named `orders` (not `order`) because `ORDER` is a reserved word in SQL.

(b) (4 marks) Foreign keys and referential constraints.

✓ **Your Answer**

Table	FK	References	ON DELETE
`orders`	customer_id	customer(customer_id)	RESTRICT
`contains`	order_id	orders(order_id)	CASCADE
`contains`	sku	product(sku)	RESTRICT

Justification:

- `orders.customer_id`: `RESTRICT`, because deleting a customer with active orders should be blocked (preserve purchase history).
- `contains.order_id`: `CASCADE`, because deleting an order should remove all its line items automatically (they have no meaning without the parent order).
- `contains.sku`: `RESTRICT`, because you should not delete a product that was sold; preserve historical references.

Understanding

The choice of `ON DELETE` action depends on business semantics, not syntax. Prof's grading rubric usually accepts any action with a reasonable justification. Cascade for dependent rows, restrict (or set null) for historical records, is the standard split.

Question 14 (10 marks) — FD Theory; Medium

Given $R = (A, B, C, D, E, F)$ with $F = \{A \to B,\ B \to CD,\ D \to E,\ AF \to B\}$.

This mirrors: *A4 Q3 + Q4 + Q6(a) (closure + candidate key + Armstrong proof).*

(a) (3 marks) Compute $(AF)^+$.

✓ **Your Answer**

Step	Closure so far
Start	$\{A, F\}$
Apply $A \to B$ (LHS $A \subseteq \{A, F\}$)	$\{A, B, F\}$
Apply $B \to CD$ (LHS $B \subseteq \{A, B, F\}$)	$\{A, B, C, D, F\}$
Apply $D \to E$ (LHS $D \subseteq \{A, B, C, D, F\}$)	$\{A, B, C, D, E, F\}$

$$(AF)^+ = \{A, B, C, D, E, F\} = R.$$

Understanding

Closure algorithm: start with the LHS; keep applying FDs whose LHS is already inside until no more attributes are added. Once the closure equals R, every attribute is functionally determined by the starting set.

(b) (3 marks) Candidate keys.

✓ Your Answer

F never appears on any RHS, so F must be in every candidate key. Start from that fact and test each $\{X, F\}$ pair:

Candidate	Closure chain	Verdict
$\{A, F\}$	$\{A, F\} \xrightarrow{A \to B} \{A, B, F\} \xrightarrow{B \to CD} \{A, B, C, D, F\} \xrightarrow{D \to E} R$	✓ superkey
$\{B, F\}$	$\{B, F\} \xrightarrow{B \to CD} \{B, C, D, F\} \xrightarrow{D \to E} \{B, C, D, E, F\}$	miss A; not SK
$\{C, F\}$	$\{C, F\}$ (no FD fires)	miss A, B, D, E; not SK
$\{D, F\}$	$\{D, F\} \xrightarrow{D \to E} \{D, E, F\}$	miss A, B, C; not SK
$\{E, F\}$	$\{E, F\}$ (no FD fires)	miss A, B, C, D; not SK

Minimality check on $\{A, F\}$: remove A and $F^+ = \{F\}$ is not a superkey; remove F and $A^+ = \{A, B, C, D, E\}$ misses F. So $\{A, F\}$ is minimal.

The only candidate key is $\{A, F\}$.

Understanding

Analogy. Think of F as a key on a keyring that only you own: no one else can hand it to you through an FD. Every candidate key must carry that key.
Takeaway. Attribute that never appears on any RHS $\Rightarrow$ must be in every CK. Pair it with each remaining attribute and closure-test.

(c) (4 marks) Prove pseudo-transitivity with Armstrong's axioms.

✓ Your Answer

Claim. If $\alpha \to \beta$ and $\beta\gamma \to \delta$, then $\alpha\gamma \to \delta$.

Proof.

Step	Assertion	Justification
1.	$\alpha \to \beta$	given
2.	$\alpha\gamma \to \beta\gamma$	augmentation of step 1 with γ
3.	$\beta\gamma \to \delta$	given
4.	$\alpha\gamma \to \delta$	transitivity of steps 2 and 3

■

Understanding

The three Armstrong axioms are: **reflexivity** ($\beta \subseteq \alpha \Rightarrow \alpha \to \beta$); **augmentation** ($\alpha \to \beta \Rightarrow \alpha\gamma \to \beta\gamma$); **transitivity** ($\alpha \to \beta,\ \beta \to \gamma \Rightarrow \alpha \to \gamma$). Pseudo-transitivity is a *derived* rule, so we must assemble it from the three primitives. Augmentation bridges from $\alpha \to \beta$ to $\alpha\gamma \to \beta\gamma$; transitivity then chains into δ.

Question 15 (15 marks) — BCNF Decomposition; Hard

Given $R = (A, B, C, D, E)$ with $F = \{AB \to C,\ C \to D,\ D \to E,\ E \to A\}$.

This mirrors: *A4 Q5 + Q6 (full BCNF decomposition with lossless + dependency-preserving check).*

(a) (4 marks) Candidate keys.

✓ **Your Answer**

B appears on no RHS, so B is in every candidate key. Test each $\{B, X\}$ by closure:

Candidate	Closure chain	CK?
AB	$\{A,B\} \xrightarrow{AB\to C} \{A,B,C\} \xrightarrow{C\to D} \{A,B,C,D\} \xrightarrow{D\to E} R$	✓
BC	$\{B,C\} \xrightarrow{C\to D} \{B,C,D\} \xrightarrow{D\to E} \{B,C,D,E\} \xrightarrow{E\to A} R$	✓
BD	$\{B,D\} \xrightarrow{D\to E} \{B,D,E\} \xrightarrow{E\to A} \{A,B,D,E\} \xrightarrow{AB\to C} R$	✓
BE	$\{B,E\} \xrightarrow{E\to A} \{A,B,E\} \xrightarrow{AB\to C} \{A,B,C,E\} \xrightarrow{C\to D} R$	✓

B alone gives $B^+ = \{B\}$, so no single attribute is a superkey; each pair above is minimal.

$$\boxed{\text{Candidate keys} = \{\ AB,\ BC,\ BD,\ BE\ \}.}$$

Understanding

Analogy. $C \to D \to E \to A \to (AB \to C)$ is a carousel: once you're on it with B, one full revolution pulls in every other attribute.
Takeaway. Any $\{B, X\}$ where $X \in \{A, C, D, E\}$ reaches R, so four CKs.

(b) (4 marks) BCNF violations.

✓ **Your Answer**

For each FD, check whether the LHS is a superkey in R (a CK or larger):

FD	LHS closure in R	Superkey?	Verdict
$AB \to C$	$(AB)^+ = R$	yes (AB is a CK)	✓ OK
$C \to D$	$C^+ = \{A, C, D, E\}$, miss B	no	**violation**
$D \to E$	$D^+ = \{A, D, E\}$, miss B, C	no	**violation**
$E \to A$	$E^+ = \{A, E\}$, miss B, C, D	no	**violation**

Understanding

Analogy. B is the "seat every CK needs"; any FD whose LHS doesn't carry B cannot be a superkey.
Takeaway. Three violations. Pick the one whose closure pulls in the most attributes as the first split target; C^+ is largest (4 attributes), so start with $C \to D$.

(c) (5 marks) Decompose into BCNF (lossless).

✓ **Your Answer**

General split rule for violator $\alpha \to \beta$: $R_{\text{left}} = \alpha^+$ and $R_{\text{right}} = (R - \alpha^+) \cup \alpha$. Iterate until every fragment is in BCNF.

Level 1 split $R = (A, B, C, D, E)$ on violator $C \to D$.

$$C_R^+ = \{A, C, D, E\} \implies R_1 = (A, C, D, E) \quad \text{and} \quad R_2 = (B, C).$$

Fragment	FDs on it	Unique CK	Status
$R_1 = (A, C, D, E)$	$C \to D,\ D \to E,\ E \to A$	$\{C\}$	violators remain
$R_2 = (B, C)$	none non-trivial	$\{B, C\}$	✓ BCNF

Recheck R_1: $C \to D$ has LHS = CK ✓; $D \to E$ has $D_{R_1}^+ = \{A, D, E\}$, missing C, so D is not a superkey $\Rightarrow$ **violation**. $E \to A$ similarly violates. Keep splitting R_1.

Level 2 split $R_1 = (A, C, D, E)$ on violator $D \to E$.

$$D_{R_1}^+ = \{A, D, E\} \implies R_{1a} = (A, D, E) \quad \text{and} \quad R_{1b} = (C, D).$$

Fragment	FDs on it	Unique CK	Status
$R_{1a} = (A, D, E)$	$D \to E,\ E \to A$	$\{D\}$	$E \to A$ still violates
$R_{1b} = (C, D)$	$C \to D$	$\{C\}$	✓ BCNF

Level 3 split $R_{1a} = (A, D, E)$ on violator $E \to A$.

$$E_{R_{1a}}^+ = \{A, E\} \implies R_{1a\alpha} = (A, E) \quad \text{and} \quad R_{1a\beta} = (D, E).$$

Fragment	FDs on it	Unique CK	Status
$R_{1a\alpha} = (A, E)$	$E \to A$	$\{E\}$	✓ BCNF
$R_{1a\beta} = (D, E)$	$D \to E$	$\{D\}$	✓ BCNF

Final decomposition (all four fragments in BCNF):

$$\boxed{(A, E), \quad (D, E), \quad (C, D), \quad (B, C).}$$

Understanding

Analogy. Think of the split as a cake cut: you always cut along the violator's LHS so the LHS attribute sits on both sides of the cut. Reattaching along the shared attribute glues the cake back without gaps (lossless).
Takeaway. At each split the intersection $R_{\text{left}} \cap R_{\text{right}}$ equals the violator's LHS; that single attribute re-joins the fragments uniquely, which is the BCNF lossless-join guarantee.

(d) (2 marks) Dependency preservation.

✓ **Your Answer**

For each FD in F, check whether a single fragment contains every attribute on both sides:

FD	Attributes located in	Preserved?
$C \to D$	both in (C, D)	✓
$D \to E$	both in (D, E)	✓
$E \to A$	both in (A, E)	✓
$AB \to C$	$A \in (A, E),\ B \in (B, C),\ C \in (C, D)$ – 3 fragments	× lost

$$\boxed{\text{Decomposition is NOT dependency-preserving; } AB \to C \text{ is lost.}}$$

Understanding

Analogy. BCNF is like organizing a toolbox by tool type: clean drawers, but if a task needs a screwdriver and a wrench together, you're digging through two drawers. 3NF keeps combo-tools intact.
Takeaway. BCNF is always lossless but may lose an FD; 3NF keeps every FD but may keep some redundancy. The prof almost always sets up a single $AB \to C$-style FD to showcase this trade-off.

Question 16 (10 marks) — Integration: Course Registration; Medium

This mirrors: *A3 Q4 + A4 Q6 combined; the standard "ER + FDs + BCNF check" closing question.*

(a) (4 marks) E-R diagram.

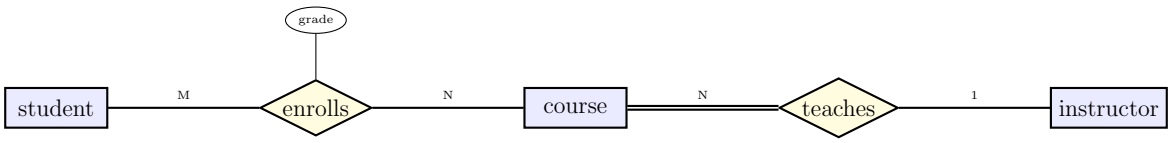
✓ **Your Answer**

Entities and keys:

- `student`(<u>student_id</u>, name, major)
- `course`(<u>course_id</u>, course_title, credits)
- `instructor`(<u>instructor_id</u>, name, department)

Relationships:

- `enrolls` between student and course, M:N, descriptive attribute `grade`. Partial participation on both sides.
- `teaches` between instructor and course, 1:N (one instructor teaches many courses; each course has exactly one instructor). Total on course (every course has an instructor).



(b) (3 marks) Functional dependencies.

✓ **Your Answer**

- `student_id` → `name, major`.
- `course_id` → `course_title, credits, instructor_id`.
- `instructor_id` → `name, department`.
- `student_id, course_id` → `grade`.

Understanding

Note the collision of the attribute name `name`: student.name and instructor.name are different columns in different entities. After mapping, they live in separate tables and do not conflict.

(c) (3 marks) BCNF check and decomposition.

✓ **Your Answer**

Map the E-R to relational tables:

- `student`(<u>student_id</u>, name, major). FD: `student_id` → name, major; LHS is the CK. **BCNF.**
- `course`(<u>course_id</u>, course_title, credits, instructor_id). FD: `course_id` → course_title, credits, instructor_id; LHS is the CK. **BCNF.**

- `instructor`(instructor_id, name, department). FD: `instructor_id` → name, department; LHS is the CK. **BCNF.**
- `enrollment`(student_id, course_id, grade). FD: (`student_id, course_id`) → grade; LHS is the CK. **BCNF.**

All four tables are already in BCNF; no decomposition is required.

Understanding

This is the "happy-path" outcome the prof likes to use as a closing question: a clean ER design with one-attribute-determines-the-rest FDs maps naturally to BCNF. The only way to trip up here is to lump student, course, and instructor into a single relation; then `course_id` → `instructor_id` would cause a BCNF violation (course_id is not the CK of the combined relation).

Appendix: Summary of Difficulty Ratings

Question	Topic	Difficulty
Q1–Q6	Concept openers	Basic
Q7	SQL (Hospital)	Medium
Q8	SQL + Views + NOT EXISTS	Medium-Hard
Q9	SQL Division / universal	Hard
Q10	RA + DDL	Medium-Hard
Q11	RA → SQL translation	Medium
Q12	ER with ISA	Medium-Hard
Q13	ER to relational	Basic-Medium
Q14	FD closure + Armstrong proof	Medium
Q15	BCNF decomposition (showpiece)	Hard
Q16	Integration: ER + FD + BCNF	Medium